

BMTM Dönem Projesi Raporu

Proje Konusu: Konu #1 - URL Shortener Service

Ders: Bulut Mimarilerinde Test Mühendisliği (MTH2526-B25)

Tarih: 26 Mayıs 2026

Grup Üyeleri

- [Ad Soyad] - [Öğrenci No] - Tech Lead
- [Ad Soyad] - [Öğrenci No] - Backend & DB
- [Ad Soyad] - [Öğrenci No] - DevOps & CI/CD

1. Giriş ve Proje Amacı

Bu projenin amacı, uzun URL'leri kısa kodlara dönüştüren, tıklama istatistiklerini takip eden ve yönlendirme sağlayan bir URL Shortener mikroservisinin baştan uca tasarlanması, kodlanması ve bulut mimari standartlarına uygun olarak test süreçlerinin işletilmesidir. Sistem FastAPI ve SQLite tabanlı olarak geliştirilmiş, Docker ve Kubernetes kullanılarak kapsayıcı mimariye uygun hale getirilmiştir.

2. Sistem Mimarisini ve Teknoloji Yığını

Mimari Yaklaşım: Projede modüler, test edilebilir ve izlenebilir bir monolit/mikroservis yaklaşımı benimsenmiştir.

- Backend:** Python 3.11, FastAPI, SQLAlchemy ORM, Pydantic
- Veritabanı:** Geliştirme/Test için SQLite, Testcontainers senaryosu için PostgreSQL
- Kapsayıcı ve Orkestrasyon:** Docker (Multi-stage build), Kubernetes (Deployment, Service, ConfigMap)
- Monitoring:** Prometheus, Grafana
- CI/CD:** GitHub Actions
- Bulut Simülasyonu:** LocalStack (AWS S3)

3. Test Stratejisi

Yazılım kalite güvencesi (QA) uçtan uca düşünülmüş ve çok katmanlı bir strateji izlenmiştir:

3.1. Unit ve Integration Testler

Pytest çerçevesi ile geliştirilen birim ve entegrasyon testleri `tests/` klasörü altında yapılandırılmıştır.

- Unit Testler:** Veritabanına bağımlı olmadan iş mantığının (kısa kod üretimi) test edilmesi.
- Integration Testler:** API endpoint'leri ve veritabanı (CRUD işlemleri) entegrasyonunun sınanması. Testcontainers ile PostgreSQL üzerinde senaryolar çalıştırılmıştır.
- Mocking & Factories:** Factory Boy ve Faker ile sahte veri üretilmiş, `conf/test.py` üzerinden izole test veritabanı oturumları sağlanmıştır.

3.2. E2E UI Testleri

`tests/e2e/test_ui.py` içerisinde Playwright kullanılarak 5 farklı uçtan uca senaryo otomatize edilmiştir. Kullanıcının arayüze girmesi, form doldurması, kısa link kopyalaması simüle edilmektedir.

3.3. API Testleri (Postman)

Kılavuzda istenildiği gibi 8 farklı API senaryosu Newman üzerinden CI/CD pipeline'da çalışacak şekilde yapılandırılmıştır. Değişken aktarımı (short_code aktarımı) başarılı bir şekilde kurulmuştur.

4. CI/CD Süreci

GitHub Actions ile otomatize edilmiş `.github/workflows/ci.yml` pipeline'ı oluşturulmuştur. Pipeline aşamaları:

1. **Linting:** Flake8 ile PEP8 stil kuralları kontrolü.
2. **Pytest & Coverage:** Testlerin çalıştırılması ve %70 coverage barajının denetlenmesi.
3. **Newman:** Postman koleksiyonunun CI ortamında koşturulması.
4. **Docker Build:** `url-shortener:latest` multi-stage imajının derlenmesi ve sağlık kontrolü (Healthcheck).
5. **Smoke Test:** Derlenen imaj ayağa kaldırılarak temel `/health` ve `/shorten` endpoint'lerinin denenmesi.

5. Performans ve Yük Testi (k6)

Sistemin 100 eşzamanlı kullanıcı (Concurrent User) altında vereceği tepki k6 ile test edilmiştir.

- Yük grafiği 4 aşamalıdır (Isınma, Yükleme, Pik Yük, Soğuma).
- Senaryo %40 URL Kısaltma, %30 Redirect üzerine kurulmuştur.
- **Sonuçlar:** p95 (95. persentil) gecikmesinin 500ms altında olup olmadığı gözlemlenmiştir. API hızlı çalışmakla birlikte veritabanı yazma kilitlenmeleri gibi performans darboğazları Grafana'dan izlenmiştir.

6. İzleme ve Gözlemlenebilirlik (Monitoring)

Uygulama `prometheus-client` kütüphanesi sayesinde `/metrics` endpoint'i üzerinden verilerini dışa açmaktadır.

- **Prometheus:** Her 15 saniyede bir metrikleri toplar (scrape).
- **Grafana:** Prometheus verilerini 6 panelli bir Dashboard üzerinde görselleştirir (URL sayısı, Redirect sayısı, 404 hatası oranı, p95 gecikme zaman serisi vb.)
- **LocalStack:** Uygulama istatistiklerini düzenli periyotlarda sahte bir AWS S3 bucket'ına atacak şekilde AWS Client entegrasyonu eklenmiştir.

7. Karşılaşılan Zorluklar ve Çözümler

- **Pip Kurulum Hataları:** Windows ortamında C++ Build tools eksikliği lokalde bazı paketlerin (pydantic-core) derlenmesini engellemiştir. **Çözüm:** Docker Multi-stage build kullanılarak standart bir Linux ortamı sağlandı ve lokal sistem bağımlılığı ortadan kaldırıldı.
- **Veritabanı İzolasyonu:** Pytest çalışırken eski verilerin yeni testleri bozması. **Çözüm:** `conf test.py` içerisinde veritabanını test başında kurup test sonunda yıkan yield-bazlı oturumlar yazıldı.

8. Sonuç

Bulut Mimarilerinde Test Mühendisliği dersi için istenen 8 kılavuz gereksinimi eksiksiz tamamlanmıştır. Kod kalitesi yüksek, kapsayıcı odaklı, modern DevOps pratiklerine uygun bir mikroservis hayata geçirilmiştir.